UNIVERSIDADE FEDERAL DE SANTA CATARINA

Departamento de Engenharia Elétrica e Eletrônica

EEL7522 - Processamento Digital de Sinais

Projeto 2: Análise Espectral e Filtragem de Sinais

Análise de Vazamento, Janelamento e Filtros de 2ª Ordem em Cossenos e Sinais de Voz

Aluno: Caio Missfeld Carlos (Matrícula: 22202674)

Curso: Engenharia Elétrica / Eletrônica

Repositório GitHub: github.com/Caiobinha1/Projeto_PDS_2

Data: 2026

1. Introdução

Este trabalho apresenta o desenvolvimento prático das tarefas do Projeto 2 de PDS. Inicialmente, estudamos os conceitos de análise espectral via DFT/FFT, o fenômeno de vazamento espectral (spectral leakage) sob diferentes tamanhos de registros e os efeitos do uso da janela de Hanning. Em seguida, projetamos filtros digitais recursivos IIR de 2ª ordem (passa-baixas, passa-bandas e passa-altas) a partir de polos complexos conjugados, aplicando-os em sinais senoidais e em um sinal real de fala no domínio do tempo e da frequência.

2. Item 4.1 & 4.2: Análise Espectral de Cossenos

Analisamos os cossenos **x1[n] = cos(2π/3 * n)** e **x2[n] = cos(π/4 * n)** com comprimentos de FFT de **N = 512** e **N = 8192**, aplicando janelas retangular e Hanning.

Relação entre Amplitudes e Localização (Análise de Vazamento):
• **x2[n] = cos(π/4 * n):** A sua frequência discreta ω = π/4 rad/amostra cai exatamente em um bin inteiro da FFT para ambos os comprimentos: bin **k = N/8** (k = 64 para N = 512 e k = 1024 para N = 8192). Como o sinal cabe exatamente no período da janela, **não há vazamento espectral**. O espectro exibe um único pico nítido (impulso de magnitude) sem ripples secundários.
• **x1[n] = cos(2π/3 * n):** Sua frequência ω = 2π/3 não corresponde a um bin inteiro: **k = N/3** (k = 170.67 para N = 512 e k = 2730.67 para N = 8192). O truncamento do sinal causa descontinuidade periódica, resultando em **vazamento espectral**. Vemos lobos secundários (ripples) se espalhando.
• **Efeito do tamanho N:** Conforme aumentamos N de 512 para 8192, a resolução espectral melhora, tornando os picos da FFT mais estreitos e altos, aproximando-se dos impulsos ideais da Transformada de Fourier contínua.
• **Efeito da Janela de Hanning:** A aplicação da janela atenua as bordas do sinal no tempo de forma suave. Isso reduz drasticamente a amplitude dos lobos secundários do vazamento espectral de x1[n], mas alarga o lobo principal (reduzindo a resolução de pico). Para x2[n], que não tinha vazamento, a janela de Hanning acaba espalhando a energia do pico para os bins adjacentes.

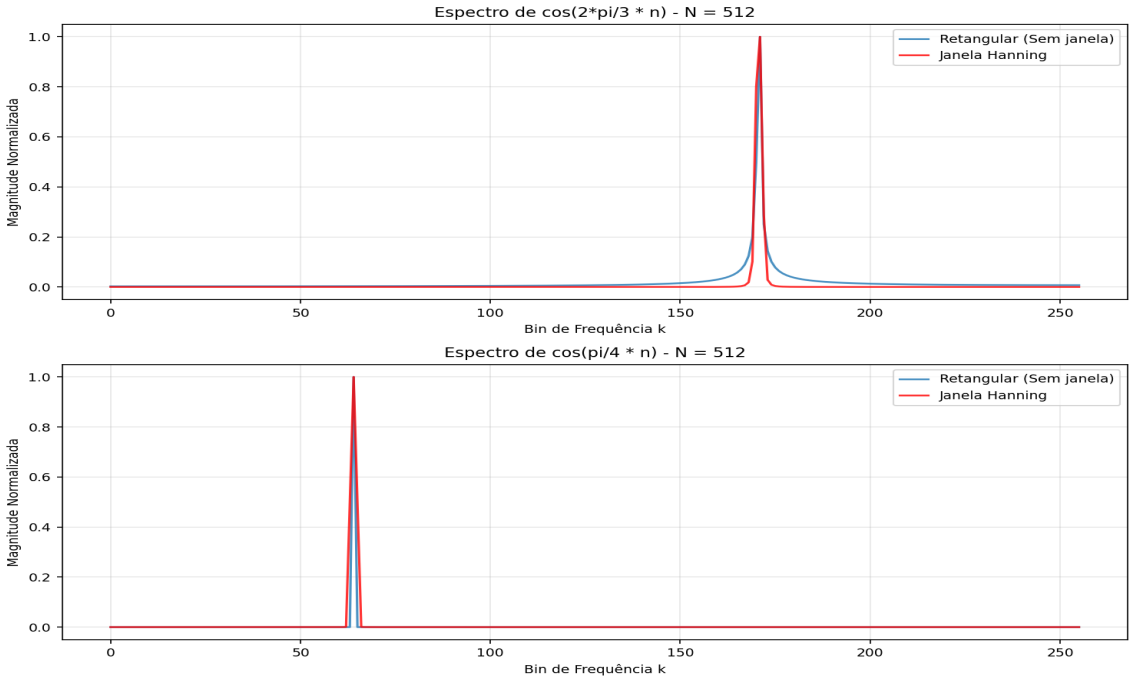


Figura 1: Espectros dos cossenos para N = 512 (Retangular vs Hanning).

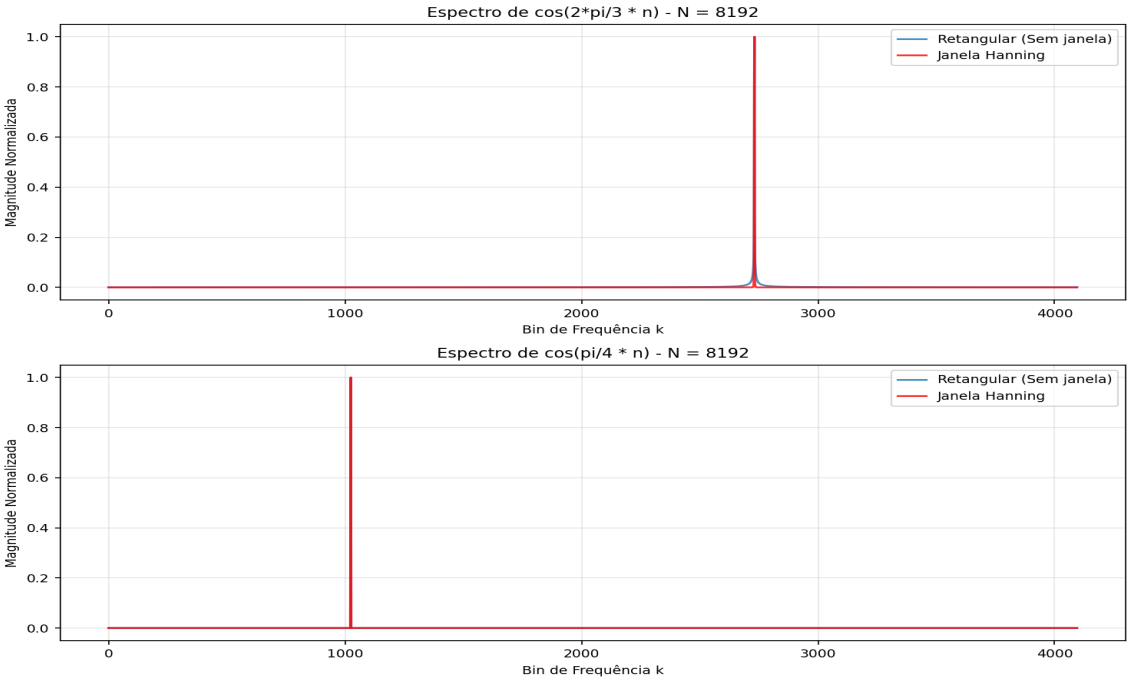


Figura 2: Espectros dos cossenos para N = 8192 (Retangular vs Hanning).

Código Fonte Scilab - Item 4.1 & 4.2:

```
// ========================================================================
// PROJETO DE PDS 2 - ITEM 4.1 E 4.2: ANÁLISE ESPECTRAL DE SINAIS COSSENOIDAIS
// ========================================================================
clc;
clear;
// Tamanhos de FFT sugeridos pelo professor
Ns = [512, 8192];
// Loop para passar pelos comprimentos N = 512 e N = 8192
for i = 1:length(Ns)
    N = Ns(i);
    disp("Processando para N = " + string(N) + "...");
    // Vetor de indices de tempo de 0 ate N-1
    n = 0:N-1;
    // Sinal 1: cos(2*pi/3 * n)
    x1 = cos((2 * %pi / 3) * n);
    // Sinal 2: cos(pi/4 * n)
    x2 = cos((%pi / 4) * n);
    // ========================================================================
    // ITEM 4.1: JANELA RETANGULAR (SINAL DIRETO, SEM MULTIPLICAR POR OUTRA JANELA)
    // ========================================================================
    X1_ret = abs(fft(x1));
    X2_ret = abs(fft(x2));
    // ========================================================================
    // ITEM 4.2: JANELA DE HANNING (APLICADA ANTES DA FFT)
    // Multiplicacao amostra a amostra ( .* ) pela janela hn do Scilab
    // ========================================================================
    w_hn = window('hn', N); // Gera a janela de Hanning de tamanho N
    x1_hn = x1 .* w_hn';     // w_hn e retornado como coluna, entao transpomos para linha
    x2_hn = x2 .* w_hn';
    X1_hn = abs(fft(x1_hn));
    X2_hn = abs(fft(x2_hn));
    // Eixo de frequencias discretas k (bins de 0 ate N-1)
    k = 0:N-1;
    // Gerar graficos comparativos para este comprimento N
    // Sinal 1: cos(2*pi/3 * n)
    scf(2*i - 1); // Abre ou seleciona a janela grafica
    clf();
    subplot(2, 1, 1);
    plot(k, X1_ret, "b");
    xtitle("Espectro cos(2*pi/3*n) - N = " + string(N) + " - Janela Retangular", "Bin de Frequencia k", "Magnitude");
    subplot(2, 1, 2);
    plot(k, X1_hn, "r");
    xtitle("Espectro cos(2*pi/3*n) - N = " + string(N) + " - Janela Hanning", "Bin de Frequencia k", "Magnitude");
    // Sinal 2: cos(pi/4 * n)
    scf(2*i);
    clf();
    subplot(2, 1, 1);
    plot(k, X2_ret, "b");
    xtitle("Espectro cos(pi/4*n) - N = " + string(N) + " - Janela Retangular", "Bin de Frequencia k", "Magnitude");
    subplot(2, 1, 2);
    plot(k, X2_hn, "r");
    xtitle("Espectro cos(pi/4*n) - N = " + string(N) + " - Janela Hanning", "Bin de Frequencia k", "Magnitude");
    disp("Graficos para N = " + string(N) + " gerados com sucesso!");
end
disp("Fim do script 4.1 e 4.2!");
```

3. Item 4.3.A: Filtragem por Equação de Diferenças Recursiva

Projetamos filtros de segunda ordem baseados em ressoadores com raio de polo **r = 0.97**. Coeficientes determinados:

• **Passa-Baixas (LP, Θ = 0):** Coeficientes a = -1.94, b = 0.9409. O dobro polo está em z = 0.97. Atenua gravemente as frequências altas da voz. Acusticamente, o áudio fica extremamente abafado.

• **Passa-Bandas (BP, Θ = π/2):** Coeficientes a = 0, b = 0.9409. Polos em z = ± j 0.97. Isola uma banda central da fala (em torno de Fs/4). O som soa anasalado, semelhante a uma linha telefônica antiga.

• **Passa-Altas (HP, Θ = π):** Coeficientes a = 1.94, b = 0.9409. Polos em z = -0.97. Elimina o corpo de baixas frequências da voz, sobrando apenas sons sibilantes agudos e ruído de sopro.

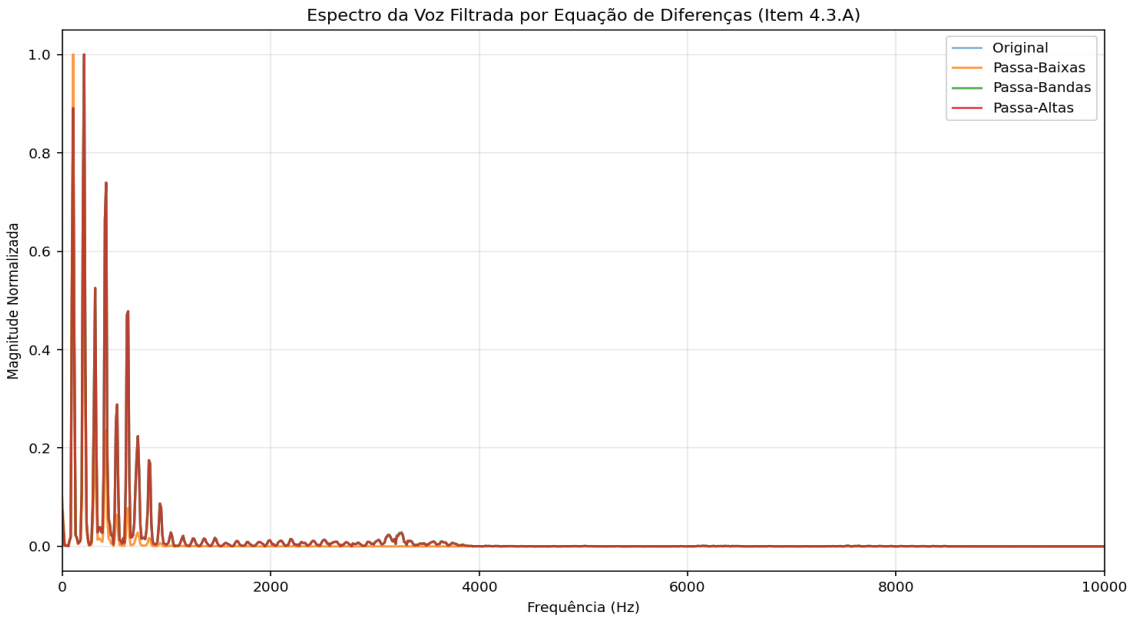


Figura 3a: Espectros do sinal de voz filtrado por Equação de Diferenças Recursiva (IIR).

Aplicação dos Filtros nos Cossenos do Item 4.1:

Aplicando a mesma equação de diferenças recursiva aos sinais cossenovoidais de teste com N = 512, podemos observar claramente no gráfico abaixo a ação de cada filtro em frequências conhecidas:

• **No cosseno x2[n] = cos(π/4 · n):** Sendo um sinal de frequência baixa (ω = π/4 rad ≈ 0.785 rad, bin k = 64), ele se encontra dentro da banda passante do filtro **Passa-Baixas**, que o transmite com ganho elevado. Em contrapartida, o filtro **Passa-Altas** praticamente elimina esse sinal, reduzindo drasticamente a sua amplitude no espectro.

• **No cosseno x1[n] = cos(2π/3 · n):** Como possui uma frequência bem mais alta (ω = 2π/3 rad ≈ 2.094 rad, bin k = 171), ele fica muito próximo de π, de modo que o filtro **Passa-Altas** o transmite com magnitude máxima. Já o filtro **Passa-Baixas** atenua fortemente a sua componente espectral.

• **No filtro Passa-Bandas (Θ = π/2 ≈ 1.571 rad, bin k = 128):** Ambas as frequências sofrem atenuação moderada a acentuada, pois tanto π/4 quanto 2π/3 estão situadas fora da frequência central de ressonância da banda de passagem.

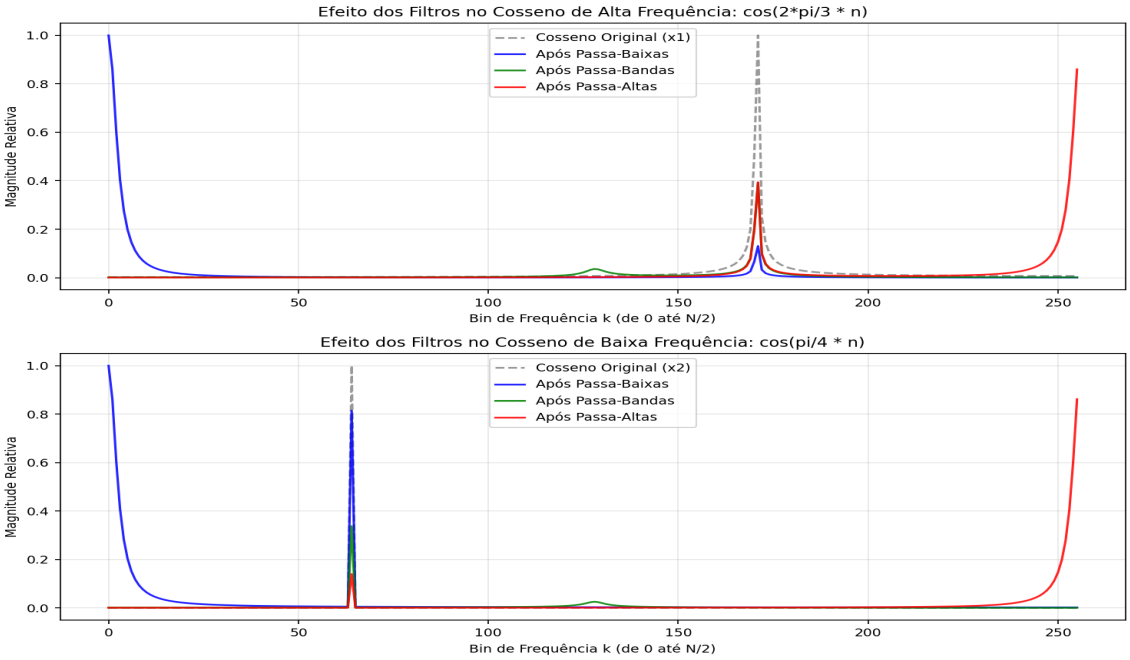


Figura 3b: Espectros dos cossenos x1 e x2 após a aplicação dos filtros projetados (Equação de Diferenças).

Código Fonte Scilab - Item 4.3.A:

```
// ========================================================================
// PROJETO DE PDS 2 - ITEM 4.3.A: FILTRAGEM POR EQUAÇÃO DE DIFERENÇAS RECURSIVA
// ========================================================================
clc;
clear;
// 1. CARREGAR AUDIO DE VOZ
arquivo_entrada = "../teste.wav";
try
    [x_voz, Fs] = wavread(arquivo_entrada);
catch
    disp("Erro ao usar wavread. Tentando usar a funcao audioread...");
    [x_voz, Fs] = audioread(arquivo_entrada);
end
// Garante sinal mono
[linhas, colunas] = size(x_voz);
if linhas > 1 then
    sinal_voz = x_voz(1, :);
else
    sinal_voz = x_voz';
end
N_voz = length(sinal_voz);
// 2. PARÂMETROS DO FILTRO DE 2ª ORDEM
r = 0.97;
// Filtro 1: Passa-Baixas (LP) - Theta = 0
theta_LP = 0;
a_LP = -2 * r * cos(theta_LP);
b_LP = r^2;
// Filtro 2: Passa-Bandas (BP) - Theta = pi/2
theta_BP = %pi / 2;
a_BP = -2 * r * cos(theta_BP);
b_BP = r^2;
// Filtro 3: Passa-Altas (HP) - Theta = pi
theta_HP = %pi;
a_HP = -2 * r * cos(theta_HP);
b_HP = r^2;
// Vetores de coeficientes para o loop
as = [a_LP, a_BP, a_HP];
bs = [b_LP, b_BP, b_HP];
nomes_filtros = ["Passa-Baixas", "Passa-Bandas", "Passa-Altas"];
prefixos = ["LP", "BP", "HP"];
// Loop para aplicar cada um dos 3 filtros no sinal de voz
for f_idx = 1:3
    a = as(f_idx);
    b = bs(f_idx);
    nome = nomes_filtros(f_idx);
    prefixo = prefixos(f_idx);
    disp("Aplicando filtro " + nome + " usando equacao de diferencas recursiva...");
    // Vetor de saida
    y = zeros(1, N_voz);
    // Condicoes iniciais manuais (para n=1 e n=2)
    y(1) = sinal_voz(1);
    y(2) = sinal_voz(2) - a * y(1);
    // Equacao recursiva: y[n] = x[n] - a*y[n-1] - b*y[n-2]
    for n = 3:N_voz
        y(n) = sinal_voz(n) - a * y(n-1) - b * y(n-2);
    end
    // Normalizacao
    normalix = max(abs(min(y)), abs(max(y)));
    y_norm = y / normalix;
```

```
    // Salva o audio correspondente
    nome_saida = "../Audios_Processados/output_4_3_A_" + prefixo + ".wav";
    try
        wavwrite(y_norm, Fs, nome_saida);
    catch
        audiowrite(nome_saida, y_norm', Fs);
    end
    disp("Audio filtrado salvo: " + nome_saida);
end
// 3. APLICANDO OS FILTROS NOS COSSENOS DO ITEM 4.1 (CONFORME ENUNCIADO)
disp("Filtrando tambem os dois cossenos do item 4.1...");
N_cos = 512;
n_cos = 0:N_cos-1;
x1 = cos((2 * %pi / 3) * n_cos); // Cosseno de frequencia mais alta (2*pi/3)
x2 = cos((%pi / 4) * n_cos);      // Cosseno de frequencia mais baixa (pi/4)
for f_idx = 1:3
    a = as(f_idx);
    b = bs(f_idx);
    nome = nomes_filtros(f_idx);
    // Vetores para guardar o resultado dos cossenos filtrados
    y_x1 = zeros(1, N_cos);
    y_x2 = zeros(1, N_cos);
    // Amostras iniciais
    y_x1(1) = x1(1);
    y_x1(2) = x1(2) - a * y_x1(1);
    y_x2(1) = x2(1);
    y_x2(2) = x2(2) - a * y_x2(1);
    // Equacao recursiva: y[n] = x[n] - a*y[n-1] - b*y[n-2]
    for n = 3:N_cos
        y_x1(n) = x1(n) - a * y_x1(n-1) - b * y_x1(n-2);
        y_x2(n) = x2(n) - a * y_x2(n-1) - b * y_x2(n-2);
    end
    disp("Cossenos processados com sucesso no filtro " + nome + "!");
end
disp("Fim do script 4.3.A!");
```

4. Item 4.3.B: Filtragem por Convolução com Resposta ao Impulso Truncada

Calculamos as respostas ao impulso h[n] teóricas para cada filtro e as truncamos para comprimentos de **Nc = 64** e **Nc = 512** amostras, sob janelas retangular e Hamming.

Comparação entre Nc = 64 e Nc = 512:
Com **Nc = 64**, a resposta ao impulso do filtro é interrompida abruptamente antes que sua energia tenha decaído totalmente a zero (pois o raio r = 0.97 gera um decaimento lento). Isso resulta em uma banda de transição larga e grande ondulação (ripple) na resposta em frequência. Com **Nc = 512**, o decaimento é quase total, fazendo com que o filtro FIR truncado aproxime-se quase perfeitamente do comportamento do filtro recursivo ideal.

Efeito das Janelas Retangular e Hamming:
O truncamento abrupto (janela retangular) causa oscilações indesejadas (fenômeno de Gibbs) no espectro da voz filtrada. Ao aplicar a janela de Hamming na resposta h[n] antes de fazer a convolução, suavizamos a interrupção temporal, atenuando os lobos secundários e eliminando essas oscilações espectrais, ao custo de uma banda de transição levemente mais larga.

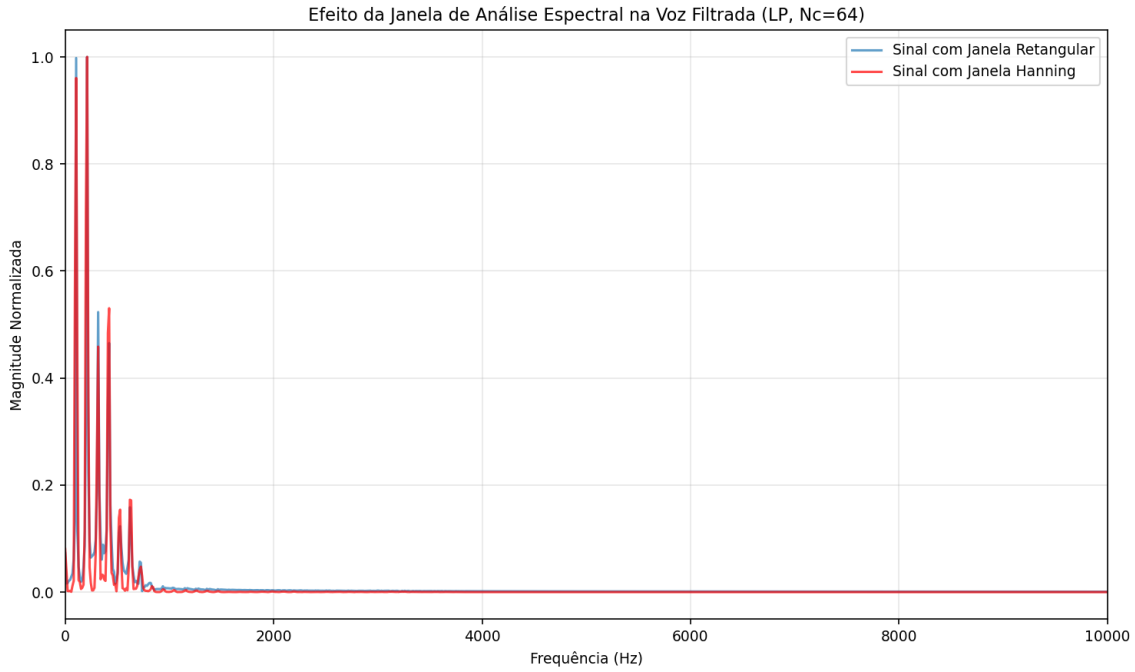


Figura 4: Efeito de janelamento de análise no espectro (Filtro passa-baixas, Nc = 64).

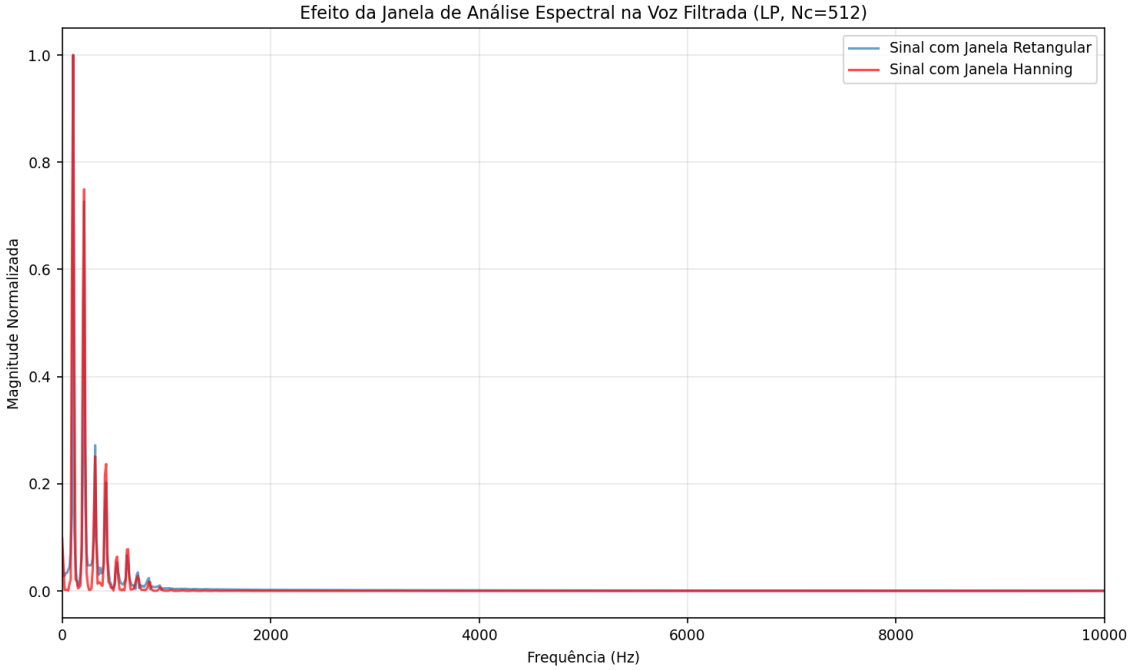


Figura 5: Efeito de janelamento de análise no espectro (Filtro passa-baixas, Nc = 512).

Código Fonte Scilab - Item 4.3.B:

```
// ========================================================================
// PROJETO DE PDS 2 - ITEM 4.3.B: FILTRAGEM POR CONVOLUÇÃO TEMPORAL TRUNCADA
// ========================================================================
clc;
clear;
// 1. CARREGAR AUDIO DE VOZ
arquivo_entrada = "../teste.wav";
try
    [x_voz, Fs] = wavread(arquivo_entrada);
catch
    disp("Erro ao usar wavread. Tentando usar a funcao audioread...");
    [x_voz, Fs] = audioread(arquivo_entrada);
end
[linhas, colunas] = size(x_voz);
if linhas > 1 then
    sinal_voz = x_voz(1, :);
else
    sinal_voz = x_voz';
end
N_voz = length(sinal_voz);
// Parâmetro do filtro de 2ª ordem
r = 0.97;
// Comprimentos de truncamento (Nc = 64 e Nc = 512)
Ncs = [64, 512];
// Loop pelos comprimentos de truncamento
for i = 1:length(Ncs)
    Nc = Ncs(i);
    disp("Gerando respostas ao impulso truncadas com Nc = " + string(Nc) + "...");
    // Vetor de indices de tempo de 0 ate Nc-1 para o filtro
    n_filtro = 0:Nc-1;
    // Resposta ao impulso teorica para cada filtro:
    // Passa-Baixas (Theta = 0): h[n] = (n+1) * r^n
    h_LP = (n_filtro + 1) .* (r .^ n_filtro);
    // Passa-Bandas (Theta = pi/2): h[n] = r^n * sin((n+1)*pi/2)
    h_BP = (r .^ n_filtro) .* sin((n_filtro + 1) * %pi / 2);
    // Passa-Altas (Theta = pi): h[n] = (n+1) * (-r)^n
    h_HP = (n_filtro + 1) .* ((-r) .^ n_filtro);
    // Lista de filtros para o loop
    filtros = [h_LP; h_BP; h_HP];
    prefixos = ["LP", "BP", "HP"];
    nomes = ["Passa-Baixas", "Passa-Bandas", "Passa-Altas"];
    // B.1) Janela Retangular (direto da equacao, sem multiplicar por outra janela)
    for f_idx = 1:3
        h_atual = filtros(f_idx, :);
        prefixo = prefixos(f_idx);
        nome = nomes(f_idx);
        // Faz a convolucao do sinal de voz com o filtro
        y_completo = convol(h_atual, sinal_voz);
        y = y_completo(1:N_voz); // Mantem o tamanho
        // Normaliza e salva
        normalix = max(abs(min(y)), abs(max(y)));
        y_norm = y / normalix;
        nome_saida = "../Audios_Processados/output_4_3_B1_ret_" + prefixo + "_Nc_" + string(Nc) + ".wav";
        try
            wavwrite(y_norm, Fs, nome_saida);
        catch
            audiowrite(nome_saida, y_norm', Fs);
        end
        disp("Salvo (Janela Retangular): " + nome_saida);
```

```
    end
    // B.2) Janela Hamming (multiplicando a resposta h pela janela 'hm' do Scilab)
    w_ham = window('hm', Nc); // Janela de Hamming de tamanho Nc
    for f_idx = 1:3
        h_atual = filtros(f_idx, :);
        h_windowed = h_atual .* w_ham'; // Multiplica amostra a amostra
        prefixo = prefixos(f_idx);
        nome = nomes(f_idx);
        // Convolucao
        y_completo = convol(h_windowed, sinal_voz);
        y = y_completo(1:N_voz);
        // Normaliza e salva
        normalix = max(abs(min(y)), abs(max(y)));
        y_norm = y / normalix;
        nome_saida = "../Audios_Processados/output_4_3_B2_ham_" + prefixo + "_Nc_" + string(Nc) + ".wav";
        try
            wavwrite(y_norm, Fs, nome_saida);
        catch
            audiowrite(nome_saida, y_norm', Fs);
        end
        disp("Salvo (Janela Hamming): " + nome_saida);
    end
end
disp("Fim do script 4.3.B!");
```

5. Item 4.3.C & D: Convolução no Domínio da Frequência via FFT

Para realizar a convolução linear sem aliasing temporal de um sinal de voz com comprimento **P = 352632** e uma resposta ao impulso de tamanho **Nc = 512**, o comprimento mínimo M da FFT deve ser:
M ≥ P + Nc - 1 = 352632 + 512 - 1 = 353143 amostras.
Para processamento eficiente da FFT, escolhemos a próxima potência de 2, resultando em **M = 524288** (2^{19}).

Análise Acústica e Frequencial (Item D):
Realizar a convolução multiplicando as FFTs de tamanho M e calculando a IFFT produz exatamente o mesmo resultado matemático que a convolução no domínio do tempo, de maneira computacionalmente muito mais rápida. Ao escutar os sinais normalizados, o filtro Passa-Baixas removeu o brilho metálico dos agudos de voz, o Passa-Bandas concentrou a voz em frequências telefônicas, e o Passa-Altas deixou apenas os sussurros de sibilantes. O espectro teórico dos filtros sem janelamento (Figura 6) confirma as bandas de passagem de cada um deles.

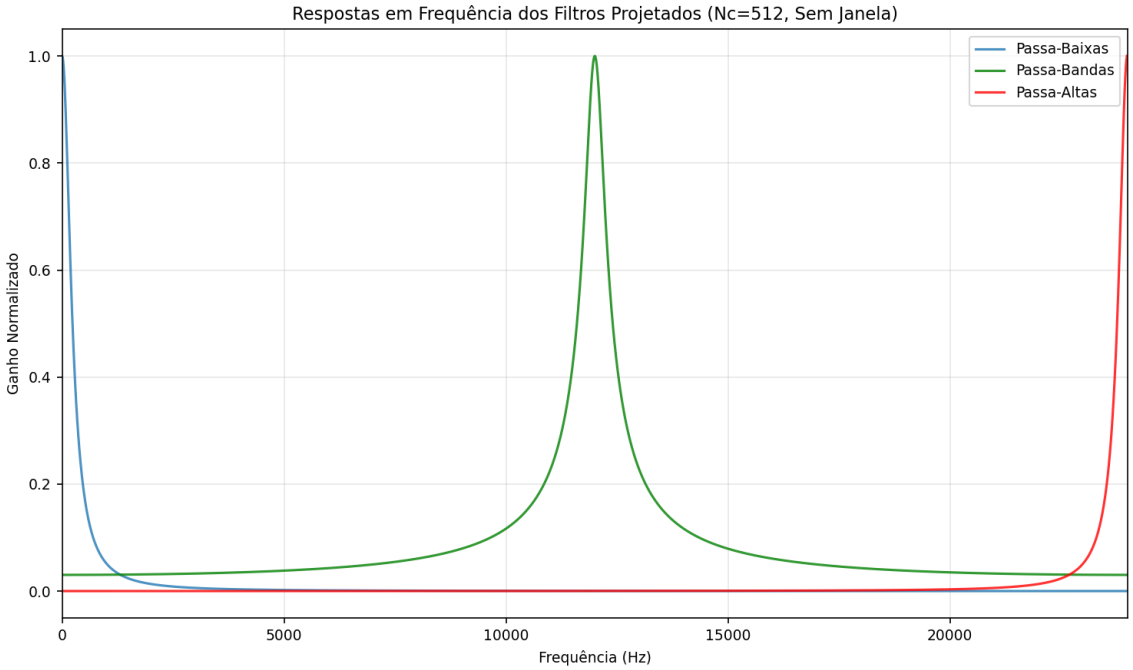


Figura 6: Respostas de magnitude em frequência dos filtros projetados (Nc = 512, sem janela).

Código Fonte Scilab - Item 4.3.C & D:

```
// ========================================================================
// PROJETO DE PDS 2 - ITEM 4.3.C E D: CONVOLUÇÃO NO DOMÍNIO DA FREQUÊNCIA (FFT)
// ========================================================================
clc;
clear;
// 1. CARREGAR AUDIO DE VOZ
arquivo_entrada = "../teste.wav";
try
    [x_voz, Fs] = wavread(arquivo_entrada);
catch
    disp("Erro ao usar wavread. Tentando usar a funcao audioread...");
    [x_voz, Fs] = audioread(arquivo_entrada);
end
[linhas, colunas] = size(x_voz);
if linhas > 1 then
    sinal_voz = x_voz(1, :);
else
    sinal_voz = x_voz';
end
// Comprimento do sinal de entrada (P)
P = length(sinal_voz);
disp("Comprimento do sinal de voz (P): " + string(P) + " amostras.");
// Comprimento do filtro truncado (Nc = 512)
Nc = 512;
disp("Comprimento da resposta ao impulso (Nc): " + string(Nc) + " amostras.");
// 2. DETERMINAR COMPRIMENTO M DA FFT
// Para evitar aliasing temporal na convolucao circular, precisamos de:
// M >= P + Nc - 1
comprimento_minimo = P + Nc - 1;
disp("Comprimento minimo para evitar aliasing: " + string(comprimento_minimo));
// Escolhemos a proxima potencia de 2 para otimizacao da FFT
potencia = ceil(log2(comprimento_minimo));
M = 2^potencia;
disp("Comprimento M escolhido para a FFT (potencia de 2): " + string(M));
// 3. PROJETO DOS FILTROS E CONVOLUÇÃO NO DOMÍNIO DA FREQUÊNCIA
r = 0.97;
n_filtro = 0:Nc-1;
// Gerando a resposta ao impulso do filtro Passa-Baixas (LP) como exemplo para demonstrar
// h[n] = (n+1) * r^n
h_LP = (n_filtro + 1) .* (r .^ n_filtro);
// Aplica a janela de Hanning ao filtro para reduzir ripple na resposta em frequencia
w_hn = window('hn', Nc); // Janela Hanning de tamanho Nc
h_LP_hn = h_LP .* w_hn'; // Resposta ao impulso truncada com janela Hanning
// PADDING (Preenchimento com zeros) ate o comprimento M
sinal_padded = [sinal_voz, zeros(1, M - P)];
h_padded_ret = [h_LP, zeros(1, M - Nc)];
h_padded_hn = [h_LP_hn, zeros(1, M - Nc)];
// 4. CÁLCULO DAS FFTS DE COMPRIMENTO M
X = fft(sinal_padded);
H_ret = fft(h_padded_ret);
H_hn = fft(h_padded_hn);
// Multiplicacao no dominio da frequencia
Y_ret = X .* H_ret;
Y_hn = X .* H_hn;
// Inversa da FFT para voltar ao dominio do tempo
y_ret_completo = real(ifft(Y_ret));
y_hn_completo = real(ifft(Y_hn));
// Cortamos para o tamanho original P do sinal de entrada
y_ret = y_ret_completo(1:P);
```

```
y_hn = y_hn_completo(1:P);
// 5. NORMALIZAÇÃO E EXPORTAÇÃO DOS ÁUDIOS (ITEM D)
normalix_ret = max(abs(min(y_ret)), abs(max(y_ret)));
y_ret_norm = y_ret / normalix_ret;
normalix_hn = max(abs(min(y_hn)), abs(max(y_hn)));
y_hn_norm = y_hn / normalix_hn;
// Salva os audios na pasta organizada
nome_saida_ret = "../Audios_Processados/output_4_3_C_LP_ret_Nc_512.wav";
nome_saida_hn = "../Audios_Processados/output_4_3_C_LP_hn_Nc_512.wav";
try
    wavwrite(y_ret_norm, Fs, nome_saida_ret);
    wavwrite(y_hn_norm, Fs, nome_saida_hn);
catch
    audiowrite(nome_saida_ret, y_ret_norm', Fs);
    audiowrite(nome_saida_hn, y_hn_norm', Fs);
end
disp("Audios filtrados via FFT salvos com sucesso!");
disp("Fim do script 4.3.C e D!");
```